ToDo

P.

The mousik package*

Celia Rubio Madrigal†

July 1, 2020

Abstract

This is a music-related package focused on writing music fragments. It works specially well with small examples, such as excerpts for theoretical articles. It is easier than other music writing methods in LaTeX because it is based on the tikz package.

Keywords

mousik, music, music theory, music notation, music writing, score, sheet, tikz diagram, bar, clef, time signature, note, symbol, rest, dot, chord, accidental, sharp, flat, natural, key

Contents

*This document corresponds to mousik v0.1, dated 2020/07/01.

†Email: celirubio.m@gmail.com

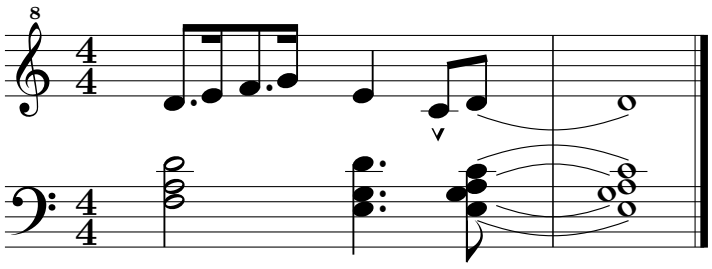
1 Introduction

There are already several ways to set music using LaTeX: musixtex, lilypond or abc, among others. Some of them are too difficult to use for not too complicated music; others do not work at all.

Based on the tikz package, the mousik package is perfect for short, one line music fragments. It may be used for longer scores, but there are more adaptable and lighter packages than this one. However, in spite of its compactness, the package is quite complete and versatile, but easy to learn and use.

2 The `mousik` environment

The way to draw a score is by means of the `mousik` environment. For longer examples, open the *examples* document.



```
\begin{mousik}[piano, piano-sep=2]
      \UpperStaff
      \Clef[octave=8] \TimeSig{4}{4}
      \MDots{*/1, , */3, } \Note{|-,=|,|-,=|}{1,2,3,4}
      \Note{4}{2}
      \Artic{^}{0} \Note{8}{0,1}
      \Tie[e=2]{1}
      \Barline
      \Note{1}{1}
      \Barline[t=|.]

      \LowerStaff
      \Clef[t=4] \TimeSig{4}{4}
      \Note{2}{13/10/8} \Space[1.5]
      \MDot{13}\MDot{9}\MDot{7} \Note{4}{13/9/7} \Space[0.5]
      \Note{8}{12/10/<9/7}
      \Tie[e=2]{12} \Tie[b=0.25,e=1.75]{10}
      \Tie[swap,b=0.25,e=1.75]{9} \Tie[swap,e=2]{7}
      \Barline
      \Note{1}{12/10/<9/7}
\end{mousik}
```

It has some options:

- *color* changes the background color. Default is white.
- *indent* indents the first line. Default is 0, in cm.
- *long* changes the maximum width of the score. Default is 100.
- *hsep* changes the horizontal separation. Default is 8. Beams and parallel dot lists follow the same separation.
- *vsep* changes the vertical separation in case of an overflow. Default is 2, in cm.
- *single* draws a one-line staff, useful for percussion examples. Notes keep the usual absolute height system; height 6 lies on the line.
- *piano* draws an upper and a lower five-line staff. Use `\UpperStaff` and `\LowerStaff` to choose which one receives the following commands. Each staff keeps its own cursor, clef, and key.
- *piano-sep* changes the distance between the two piano staves. Default is 1.6, in cm.
- *piano-bars* chooses whether barlines in piano mode are `joined` across both staves or `separate`. Default is `joined`. The command advances only the active staff.
- *tikz* allows extra options for the outside `tikz` environment.

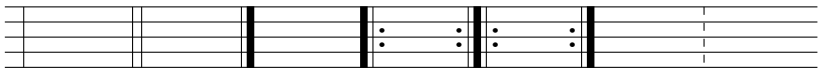
1.resize 2.compatibility issues

3 Available commands

3.1 `\Barline`

The `\Barline` command prints a bar on the score. It has some options:

- *t* (type) is the type of bar. Default is `|`, the single bar.



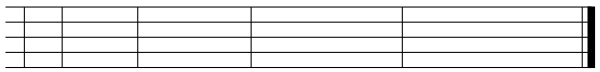
`|` `||` `|.` `|:` `:|:` `:|` `:` `(\Space)`

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

The only way to end a line in the score is with a bar or a space. Any other way may cause visual errors (see subsection 4.1).

3.2 \Space

The `\Space` command is not only used when no bar is needed. It may also be used to input *extra* space between symbols. The default space is 1, in centimeters, —which is the default space between symbols in general, so it gets added— but it may be any number (see subsection 4.2).

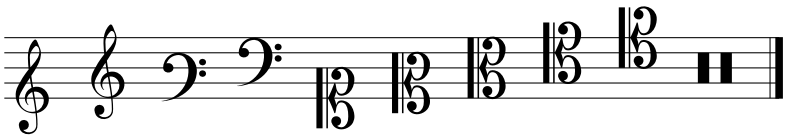


`[-0.5]` `[0]` `[0.5]` `[1]` `[1.5]`

3.3 \Clef

The `\Clef` command might be used at any moment on the score, although they are automatically printed if an overflow happens (see subsection 4.1). It has some options:

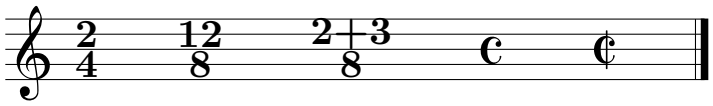
- *octave* prints an octave eight on the clef. If it is an 8, it prints it above the clef; if it is a -8, it prints it below the clef. Default is 0.
- *t* (type) is the type of clef. Default is 2 (the Treble Clef).
- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.



1 2 3 4 11 12 13 14 15 0

3.4 \TimeSig

Time signatures might be used at any moment. `\TimeSig{2}{4}` will draw a 2 by 4 time signature. Any number or symbol can be written inside the brackets. Also `\TimeSig{C}{}` will produce a *common time* symbol and `\TimeSig{C|}{}` will produce an *alla breve* symbol.



It has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

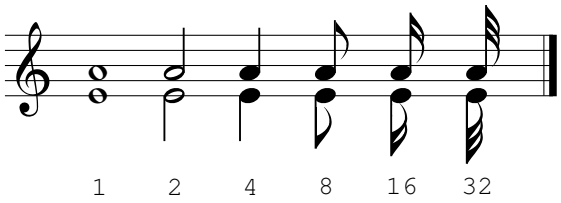
3.5 Notes and rests

3.5.1 `\Note`

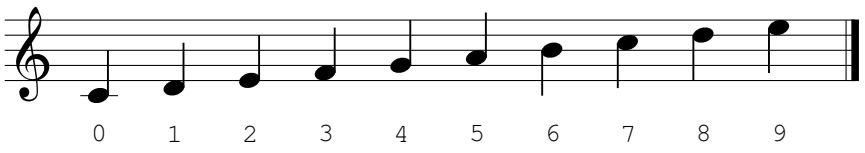
The `\Note` command prints a note on the score. It accepts 2 compulsory arguments, as well as some options:

- *beam* chooses the width of the beam. Default is 1, in mm.
- *head* chooses the type of head. Default is . (round), but also x (cross) and o (hollow) are possible.
- *stem* chooses the height of the stem. Default is 5, in mm.
- *swap* prints the element upside down. Default is false, which means up if the height is less than 6, down otherwise.
- *tuplet* prints the number of a tuplet. The written number is calculated by default, but it can be assigned optionally: `tuplet=5`.
- *tuplet-bracket* draws a bracket around the tuplet number. Default is false.
- *tuplet-xshift* moves only the tuplet marking horizontally. Default is 0, in cm.
- *tuplet-yshift* moves only the tuplet marking vertically. Default is 0, in cm.
- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

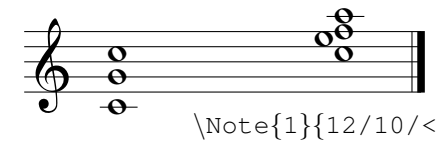
The first compulsory argument is the duration of the note.



The second argument is the absolute height. Note that the scale is in milimeters. For example, these are the notes from 0 to 9:



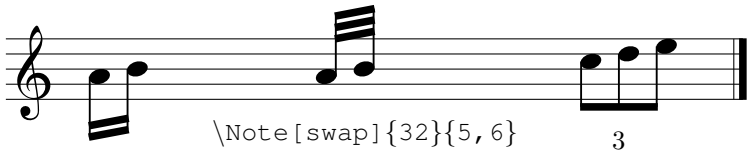
Heights separated by / correspond to chords: they belong to the same note event and are printed at the same x-position. Sometimes two heads need to be displaced to fit in a chord. For this, prefix the head that should move left with <.



```
\Note{1}{12/10/<9/7}
```

```
\Note{1}{0/4/7}
```

The \Note command also prints several notes tied together with a beam, if the second argument is a list of heights separated by commas. Two notes will be tied together with an oblique beam, but three or more will have a straight beam.

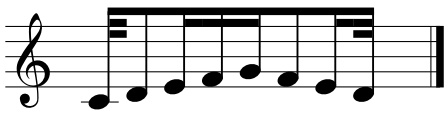



```
\Note[swap]{32}{5,6}
```

```
\Note[stem=6]{16}{5,6}
```

```
\Note[tuplet]{8}{7,8,9}
```

Notes with different durations that are tied together have a different syntax. Instead of using 8, 16 and 32 for the duration, each note has to be referenced with a −, a = or a ≡, respectively, and additional | might be needed for half the beam.

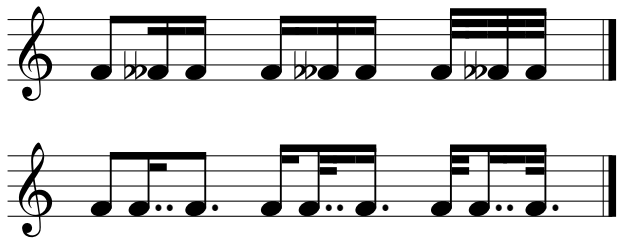


```
\Note{|≡,-,|=,=,=|,-,|=,≡|}{0,1,2,3,4,3,2,1}
```

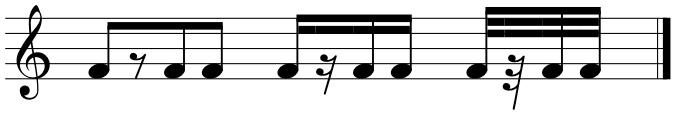
To be able to write the ≡ (which is the Unicode character U+2261) as a symbol on the .tex file, I usually copy and paste it wherever I need it. Nonetheless, an alternative keyboard-friendly input that is equally valid is using /// instead.

Additionally, spaces between notes from the same group might be useful —the default space is 0.5, in cm.

A space of 0.75 is obtained when a ! is placed on both lists. A possible usage is when dealing with wide accidentals (see section 3.6.1) or double dots (see section 3.7.1).

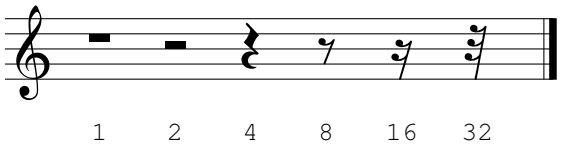


A space of 1 is obtained when a space is placed on both lists. A possible usage is when dealing with rests (see section 3.5.2).



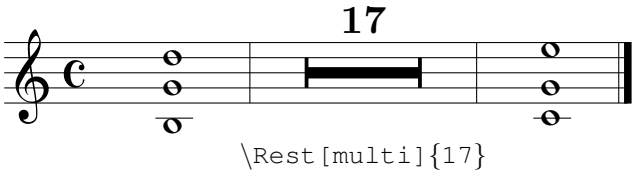
3.5.2 \Rest

The `\Rest` command prints a rest. It accepts its duration as an argument: 1, 2, 4, 8, 16 or 32.



It has some options:

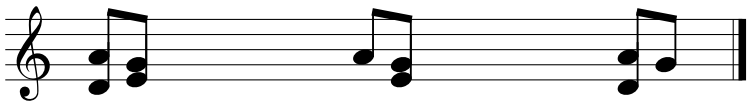
- *multi* transforms the rest into a multimeasure rest —where the rest lasts an arbitrary amount of bars.



- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

3.5.3 \Back

The \Back command moves the horizontal cursor backwards. Its optional argument is the number of note positions to step backwards, with 1 as default. It is useful for superimposing independent voices, as well as for more complicated chords.



```
\Note{8}{5,4}
  \Back[2]
\Note{4}{1,2}
```

```
\Note{8}{5,4}
   \Back
\Note{4}{2}
```

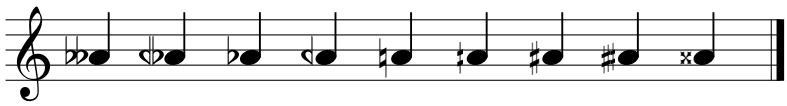
```
\Note{8}{5,4}
  \Back[2]
\Note{4}{1}
  \Space[1]
```

3.6 Keys and accidentals

3.6.1 \Acc

The Acc command prints an accidental before a note. It accepts 2 arguments.

The first argument is the type of accidental: a number ranging from -2 to 2. The flat ♭ symbol might also be called with a b and the sharp ♯ symbol might be called with \#.**3.only # ?**



```
-2  -1.5  -1  -0.5  0  0.5  1  1.5  2
          b                \#
```

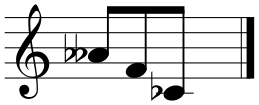
The second argument is the height of the symbol, which correspond to those of the notes.

It also has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

- *yshift* moves the element vertically (y axis). Default is 0, in cm.

When there are several notes, the \Accs command might be useful. It accepts as an argument a list of pairs: type/height. Where there is no accidental, the spot shall be left empty.

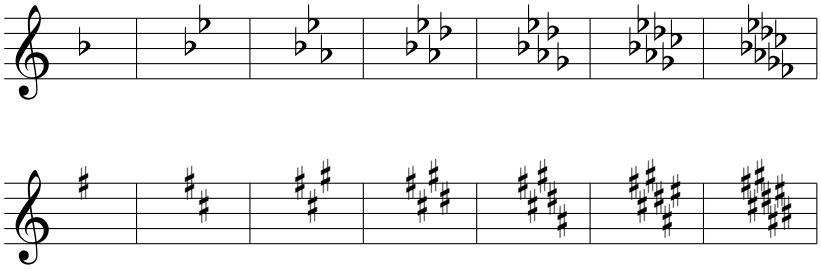


```
\Accs{-2/5, ,-1/0}
\Note85,3,0
```

3.6.2 \Key

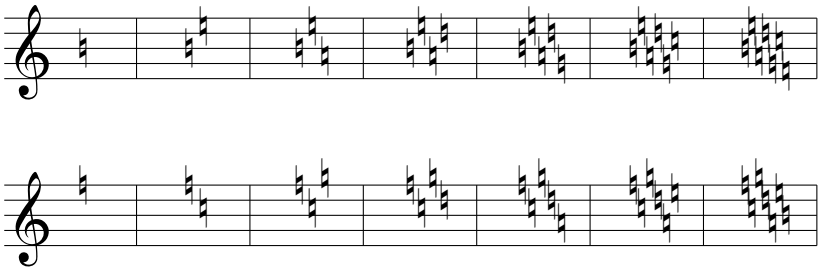
The \Key command prints the key at any moment on the score, although they are automatically printed if an overflow happens (see subsection 4.1).

It accepts an argument: a positive whole number from 1 to 7 for sharps ♯, and a negative whole number from -1 to -7 for flats ♭.

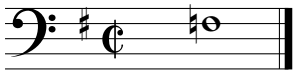


3.6.3 \NoKey

4.Mixed Keys The \NoKey command is the same as the \Key command, but it prints naturals ♮ instead of sharps or flats.



Be careful: the \Key and \NoKey commands are affected by the current clef! Nonetheless, other commands such as \Note or \Acc are not affected and need absolute height values.



```
\begin{mousik}
     \Clef[t=4]
     \Key{1}
     \TimeSig{C|}{}

     \Acc{0}{8}
     \Note{1}{8}
     \Barline[t=|.]
\end{mousik}
```

3.7 Other symbols

3.7.1 \MDot

The \MDot command prints any amount of dots in front of a note or rest. It accepts a compulsory argument: the height of the dot, which corresponds to those of the notes.

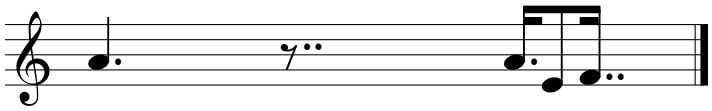
It has some options:

- *t* (type) accepts ∗∗ or .. to produce 2 dots. The default is ∗.

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

- *yshift* moves the element vertically (y axis). Default is 0, in cm.

When there are several notes, the \MDots command might be useful. It accepts as an argument a list of pairs: symbols/height, where symbols might be ∗ or ∗∗. Where there are no dots, the spot shall be left empty.



```
  \MDot{5} \MDot[t= **]{7}\MDots{*/5, , **/3}
\Note{4}{5}   \Rest{8}   \Note{|=,-,=|}{5,2,3}
```

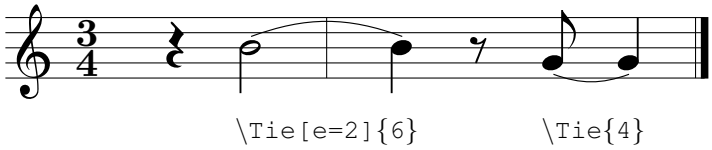
3.7.2 \Tie

The \Tie command draws a tie between two notes. It has a compulsory argument, the height of both notes, and some options:

- *angle* changes the angle in which it is drawn. Default is 20, in degrees.

- *b* (beginning) changes the starting point. Default is 0, in cm.

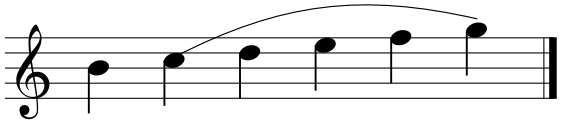
- *e* (end) changes the ending point. Default is 1, in cm.



- *swap* prints the element upside down. Default is false, which means like a *u* if the height is less than 6, and like an *n* otherwise.
- *width* changes the line width. Default is 0.15, in mm.
- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

3.7.3 The `Slurred` environment

The `Slurred` environment draws a slur between the notes placed inside. It has two arguments: the heights of both beginning and end notes.

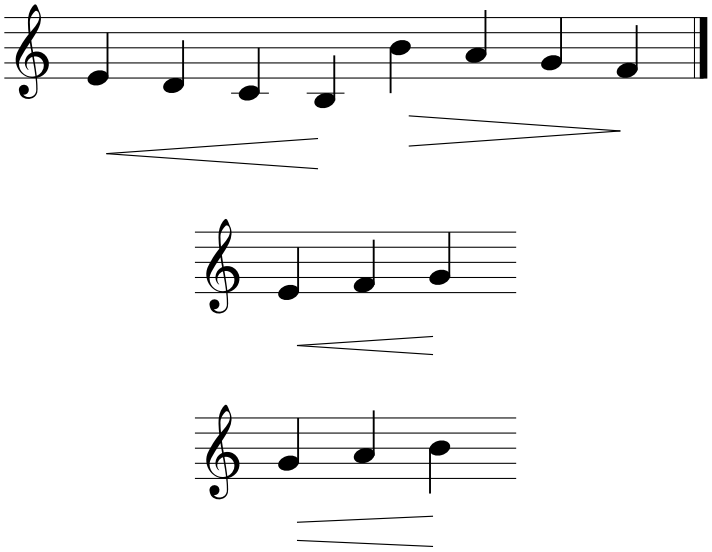


3.7.4 The `Crescendo` and `Decrescendo` environments

The `Crescendo` and `Decrescendo` environments draw hairpins between the notes placed inside. Their options are:

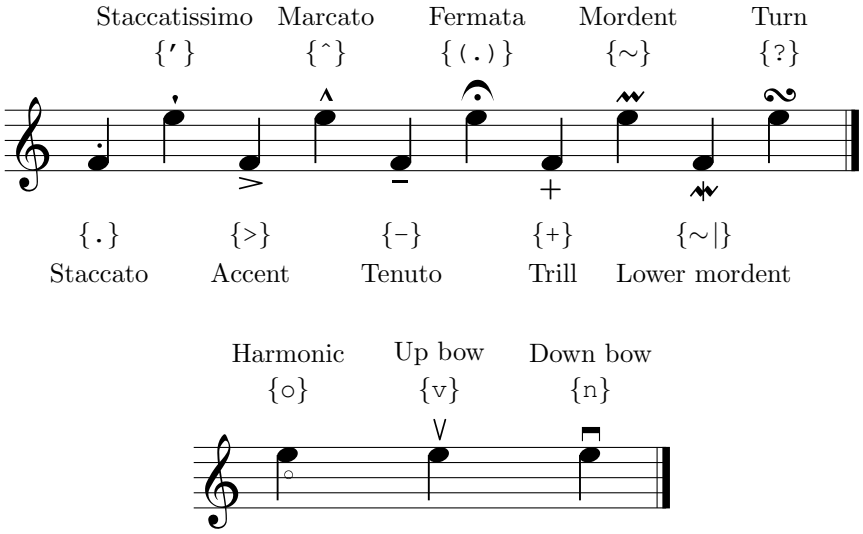
- *height* moves the centre of the hairpin vertically. Default is 0.
- *begin* and *end* set the full opening of the hairpin at its beginning and end, in cm. A crescendo defaults to `begin=0,end=0.4`. A decrescendo defaults to `begin=0.4,end=0`.
- *b* and *e* move the beginning and ending points horizontally from the first and last heads. Defaults are 0.1 and -0.1, in cm.
- *width* changes the line width. Default is 0.15, in mm.
- *xshift* moves the hairpin horizontally. Default is 0, in cm.
- *yshift* moves the hairpin vertically. Default is 0, in cm.

A hairpin that continues after a line break should be written as two independent pieces. Give the first piece a chosen *end* opening and use the same value as *begin* on the next line.



3.7.5 \Artic and \Symbol

The \Artic command prints an articulation symbol on a successive note. It accepts two arguments. The first one is the type of articulation.



The second one is the height of the symbol, which corresponds to those of the notes —but it might or might not be the height of the corresponding note.

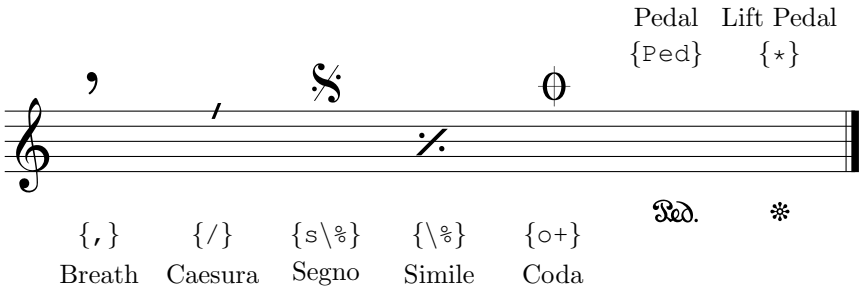
It has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

- *yshift* moves the element vertically (y axis). Default is 0, in cm.

When there are several notes, the `\Artics` command might be useful. It accepts as an argument a list of pairs: symbols/height. Where there are no symbols, the spot shall be left empty.

The `\Symbol` command is similar to the `\Artic` command, but it only accepts the type of symbol and not the height. These are symbols that have a fixed position on the score.



It also has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

- *yshift* moves the element vertically (y axis). Default is 0, in cm.

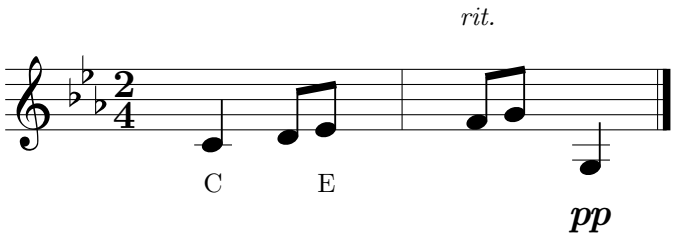
3.8 `\Text` and `\Dynamics`

The `\Text` command allows the printing of any text string. It has a compulsory argument: the text to be shown. It also has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

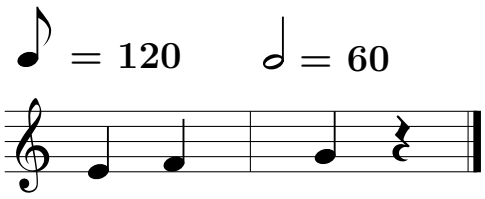
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

To write anything in the usual font of dynamics, the `\Dynamics` command will work the same: `\Dynamics[yshift=-0.5]{pp}`. **5.dynamics wout yshift peta**



3.9 \Tempo

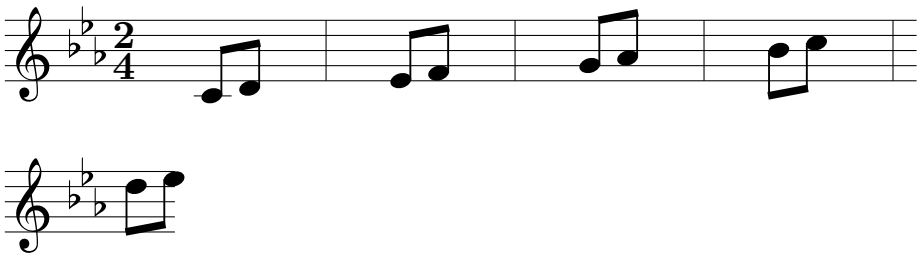
The \Tempo command prints a metronome tempo indicator. It has two arguments: the duration of the note and the numerical tempo which will be printed. It has the same options as the \Text command. An optional *yshift* of 2.5 is recommended.



4 Tips and observations

4.1 Overflow of lines

The mousik package is thought to be used for small fragments of music. The environment has a maximum length allowed —which might be changed with the *length* option— and whenever it is exceeded an overflow happens. The line is broken at that exact point and, if it does not coincide with a bar, something like this may happen:



The best way to avoid the error is by introducing extra spaces manually. The overflow will also automatically print the previous clef and key.

4.2 How do spaces work?

Spaces serve as the manual progression of an invisible cursor during the printing.

Most elements are separated by a distance of 1 cm, so a \Space command that has an argument smaller than -1 will actually step backwards. A \Space with a 0 as the argument does nothing.

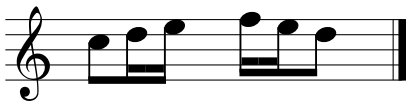
The \Space command is truly relevant in this package. Not a lot of logic is internally implemented, so the user is supposed to adjust everything at their

convenience.

This is not to say that the package becomes too cumbersome to manage. For the general use I would say it is extremely user-friendly.

4.3 What if it does become too cumbersome: your own commands

If a concrete rhythm is constantly used on a music piece, but the commands become increasingly long and tedious, the user might want to summarize it by creating their own vocabulary.



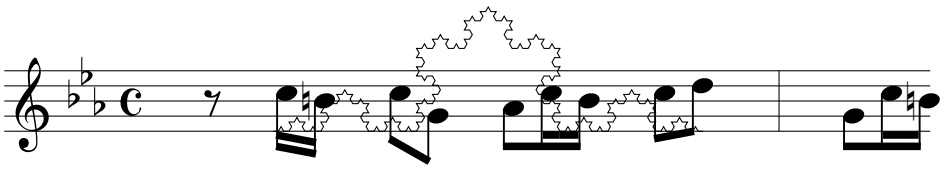
```
\newcommand{\ttc}[2][]{\Note[#1]{|-,|=,=|}{#2}}
\newcommand{\tct}[2][]{\Note[#1]{|=,=|,-|}{#2}}

\begin{mousik}
      \Clef
      \ttc{7,8,9}
      \tct{10,9,8}
      \Barline[t= |.]
\end{mousik}
```

4.4 On your own: working with the tikz package

As stated in the introduction (section 1), the mousik package is based on the tikz package. In fact, the `mousik` environment is a disguised version of a `tikzpicture` environment, but with some variables and a printed staff.

This implies two separate things:

1. The mousik environment admits other commands inside, not just the commands explained in this text. For example, I might draw nodes, lines, or even a fractal behind my score:



2. The commands explained here might be used in a `tikzpicture` environment, and will appear with no staff. However, the command `\Init` should be used at the beginning to avoid initialization errors.